hexens x OXN

Security Review Report for OXN

August 2026

Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
    - Security Review Lead
    - Scope
    - Changelog
4. Severity Structure
    - Severity characteristics
    - Issue symbolic codes
5. Findings Summary
6. Weaknesses
    - Biased work estimator inflates difficulty and slows emission relative to mintInterval
    - Incorrect BSC and Polygon block times accelerate the emission schedule
    - Difficulty readjustments can be gamed by bursting-and-idling
    - Batch Proofs Are Incompatible with the Batch Claim Path

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This report presents the results of the security review of OXN, a token protocol that combines proof-of-work mining with zero-knowledge proofs for reward claims. The assessment covered the protocol's smart contracts and associated zero-knowledge claim implementation, including mining difficulty adjustment, token emission, reward accounting, and individual and batch claims.

During our review, we identified two medium-severity and two low-severity issues. The medium-severity findings concerned work estimation and chain-specific timing assumptions that could cause token emissions to deviate from the intended schedule. The low-severity findings involved difficulty adjustment behavior under alternating bursts of mining and inactivity, and proof-format incompatibilities that prevented batch claims from completing. No critical or high-severity vulnerabilities were identified within the reviewed scope.

All reported issues were fixed by the development team. The resulting changes improved the reliability of the mining and reward claim mechanisms and strengthened the overall security and code quality of the reviewed implementation.

3. Security Review Details

Review Led by

Jahyun Koo, Lead Security Researcher

Scope

The analyzed resources were sent in an archive with the following SHA256 hash:

ccb95b198fc3c47c7dc41374472ab64c8d60f7cfb094282bce359c96eff55230

The issues described in the report were fixed in the following version (SHA256 hash):

4eb982ad210009bb4c97980e44b2fe4c752d913029389e80e7cfab2a6e57b346

Changelog

10 August 2026	Audit start
17 August 2026	Initial report
8 September 2026	Revision received
10 September 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

1. Impact of the vulnerability
2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

- **Severity Characteristics**

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical

Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.

High

Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium	Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.
Low	Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.
Informational	Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

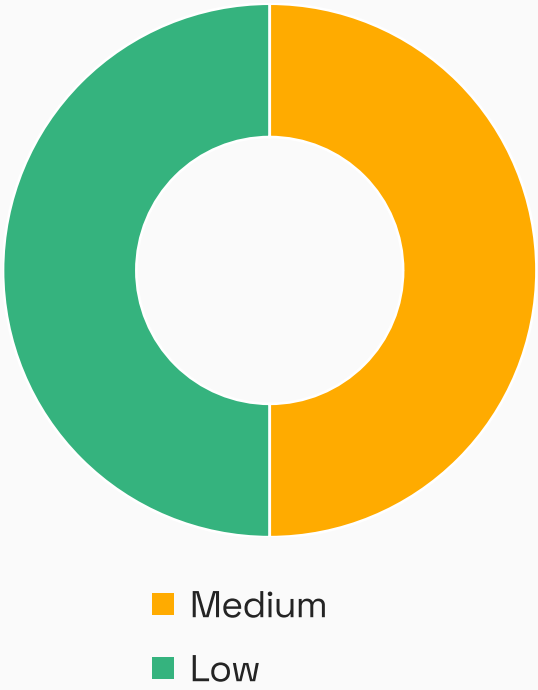
- **Issue Symbolic Codes**

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

Severity	Number of findings
Critical	0
High	0
Medium	2
Low	2
Informational	0
Total:	**4**





6. Weaknesses

This section contains the list of discovered weaknesses.

OXN1-1 | Biased work estimator inflates difficulty and slows emission relative to mintInterval

Fixed

Severity:	Probability:	Impact:
Medium	Likely	Medium

Description:

MiningEngine retargets difficulty from accumulated **_power** over each **readjustment** window. On each accepted mint it credits **_power** with the realized inverse-hash score of the solution rather than the expected work implied by the active target.

```
function _hashToDifficulty(bytes32 digest) internal pure returns (uint256) {
    uint256 val = uint256(digest);
    if (val == 0) return 0;
    return type(uint256).max / val;
}
```

```
function _executeMint(address minter, uint256 nonce, bytes32 digest, uint256 epochId, uint256 root) internal
virtual
{
    ...
    uint256 diff = _verifyDigest(epochId, minter, nonce, digest);
    ...
    _power = (type(uint256).max - diff < _power) ? type(uint256).max : _power + diff;
    ...
}
```

At readjustment, **_readjust** scales that total by **mintInterval / blocksPassed**, steps **_target** toward the result by at most about ±5% per interval, then clears _power.

```
function _readjust() internal virtual {
    uint256 blocksPassed = block.number - uint256(_lastReadjustBlock);
    uint256 step = Math.max(_target / 20, 1);
    uint256 adjustedPower = blocksPassed > 0 ? Math.mulDiv(_power, _mintInterval, blocksPassed) : 0;
```

```
        uint256 diff = Math.max(adjustedPower, _minDifficulty);
        if (diff > _target) {
            uint256 increasedTarget = _target > type(uint256).max - step ? type(uint256).max : _target + step;
            _target = Math.min(diff, increasedTarget);
        } else {
            _target = Math.max(diff, _target - step);
        }
        _lastReadjustBlock = uint48(block.number);
        _power = 0;
    }
```

A success under target T represents about T expected attempts, but the contract credits **maxUint / digest**.

Why the estimator is biased (toy scale): Replace the full hash range with 1000 and take target T = 10, so a digest succeeds when 1000 / digest ≥ 10 (i.e. digest ≤ 100).

- Expected attempts per success remain about T = 10.
- Conditional on success, digest is roughly uniform on 1..100, and the credited score is 1000 / digest.
- Average credited score is then about
  $1/100 \sum_{d=1}^{100} 1000/d \approx 51.9$, not 10.
- The same structure applies on-chain with type(uint256).max in place of 1000: accepted scores are heavy-tailed, and their mean substantially exceeds T.

Remediation:

- Credit each accepted solution with bounded expected work under the active target (e.g. the target itself), not the realized inverse-hash score.
- Ensure **_readjust** stays directionally neutral when realized cadence equals **mintInterval**.
- If realized scores are kept, cap each contribution near the active target.

OXN1-2 | Incorrect BSC and Polygon block times accelerate the emission schedule

Fixed

Severity:	Probability:	Impact:
Medium	Likely	Medium

Path:

src/MiningEngine.sol:_canReadjust, _executeMint, _readjust#L85-L86, L234-L235, L294-L347

Description:

MiningEngine lets authorized miners submit valid PoW solutions and earn rewards. Its difficulty controller aims for one accepted solution every **_mintInterval** blocks, while each accepted solution advances **_mintCount** and reduces future rewards along the emission curve.

The configured block intervals should correspond to the intended 600-second mint cadence. This keeps the reward schedule roughly consistent across supported chains and commits about 50% of the token cap after one planned half-life of 730.5 days.

However, the BSC and Polygon configurations use outdated block times. BSC is configured for 200 blocks based on three-second blocks, but its current 0.45-second blocks make the actual mint target about 90 seconds. Polygon is configured for 300 blocks based on two-second blocks, while 1.5-second blocks make the target about 450 seconds.

This can be seen in **script/config/emission.json**:

```
{ "name": "BSC", "chainId": 56, "mintIntervalBlocks": 200, "secondsPerBlock": 3 },
{ "name": "Polygon", "chainId": 137, "mintIntervalBlocks": 300, "secondsPerBlock": 2 }
```

The controller only looks at **block.number**, so it cannot account for this difference:

```
uint256 blocksPassed = block.number - uint256(_lastReadjustBlock);
uint256 adjustedPower = Math.mulDiv(_power, _mintInterval, blocksPassed);
```

These values are immutable once deployed. At the configured cadence, BSC advances the emission curve about 6.67 times faster than intended and Polygon about 1.33 times faster. By the planned first half-life, roughly 99.02% of the cap can be committed on BSC and 60.31% on Polygon instead of 50%. The cap still holds, but early miners receive rewards that were meant for later years.

Remediation:

We recommend to update the deployment parameters to match current block times.

OXN1-3 | Difficulty readjustments can be gamed by bursting-and-idling

Fixed

Severity:	Probability:	Impact:
Low	Rare	Medium

Path:

src/MiningEngine.sol:_readjust#L334-L346

Description:

Authorized pools use **mine()** to submit valid PoW solutions and receive reward commitments. Each solution adds its measured work to **_power**, and **mine()** or the permissionless readjust() function eventually calls **_readjust()** to update the difficulty based on the work seen during the last window.

The difficulty controller should adapt to the pool's actual production rate. If a pool submits far more solutions than expected, that extra work should keep pushing difficulty upward over later windows until the configured mint cadence is restored.

However, **_readjust()** limits each change to 5% and then clears **_power**, even when the observed work calls for a much larger increase:

```
_target = Math.min(diff, _target + step);
// ...
_power = 0;
```

This throws away all adjustment pressure above the 5% cap. Starting at the minimum difficulty **T**, a busy window can therefore move the target only to **1.05T**, regardless of how many solutions were submitted. One idle window then reduces it to **0.9975T**, which the minimum-difficulty clamp restores to exactly **T**. With the production setting where the readjustment window is 1.5 mint intervals, two windows are meant to produce three mints. A pool that produces more than three in the busy window can beat that adjustment while keeping difficulty between only **T** and **1.05T**.

For example:

1. A pool with **MINTER_ROLE** submits at least four valid solutions during one readjustment window.
2. At the boundary, it calls **readjust()**. The target rises by at most 5%, and the remaining adjustment pressure is discarded.
3. The pool submits nothing for the next window, then calls readjust() again. The target returns to minDifficulty.

4. The pool repeats the cycle, advancing the mint-count-based reward schedule faster and taking reward opportunities away from later miners.

```
function _readjust() internal virtual {
    uint256 blocksPassed = block.number - uint256(_lastReadjustBlock);
    uint256 step = Math.max(_target / 20, 1);
    uint256 adjustedPower = blocksPassed > 0 ? Math.mulDiv(_power, _mintInterval, blocksPassed) : 0;
    uint256 diff = Math.max(adjustedPower, _minDifficulty);
    if (diff > _target) {
        uint256 increasedTarget = _target > type(uint256).max - step ? type(uint256).max : _target + step;
        _target = Math.min(diff, increasedTarget);
    } else {
        _target = Math.max(diff, _target - step);
    }
    _lastReadjustBlock = uint48(block.number);
    _power = 0;
}
```

Remediation:

Preserve excess minting pressure when a target change is capped. An idle window should not fully erase the signal from an earlier overproductive window.

OXN1-4 | Batch Proofs Are Incompatible with the Batch Claim Path

Fixed

Severity:	Probability:	Impact:
Low	Rare	Low

Description:

A standard BN254 Groth16 proof contains the **A**, **B**, and **C** points. When encoded for Solidity, these points occupy eight 32-byte words:

```
A: 2 words
B: 4 words
C: 2 words
Total: 8 × 32 = 256 bytes
```

The batch prover and vault interface are designed around this 256-byte format.

```
solidity := proofBN254.MarshalSolidity()
if len(solidity) != 32*8 {
    return nil, fmt.Errorf("unexpected proof encoding length: %d", len(solidity))
}
```

However, the byte and SHA-256 operations in the **BatchMerkleClaimCircuit** cause the pinned gnark compiler to add one Pedersen commitment.

```
func (circuit *BatchMerkleClaimCircuit) Define(api frontend.API) error {
 poseidon, err := poseidon2.NewPoseidon2FromParameters(api, BatchPoseidon2Width,
BatchPoseidon2FullRounds, BatchNumPoseidon2Rounds)
 if err != nil {
  return err
 }

 for i := 0; i < BatchSize; i++ {
  inner1 := poseidon.Compress(circuit.Claims[i].Wallet, circuit.Claims[i].Balance)
  inner2 := poseidon.Compress(circuit.ChainId, circuit.ContractAddr)
  leaf := poseidon.Compress(inner1, inner2)

  node := leaf
  for j := 0; j < BatchTreeDepth; j++ {
```

```
  left := api.Select(circuit.MerkleHelpers[i][j], circuit.MerklePaths[i][j], node)
  right := api.Select(circuit.MerkleHelpers[i][j], node, circuit.MerklePaths[i][j])
  node = poseidon.Compress(left, right)
 }
 api.AssertIsEqual(node, circuit.Root)
}

var hash frontend.Variable = 0
for i := 0; i < BatchSize; i++ {
 hash = poseidon.Compress(hash, circuit.Claims[i].Wallet)
 hash = poseidon.Compress(hash, circuit.Claims[i].Balance)
}
api.AssertIsEqual(hash, circuit.ClaimsHash)

bytesAPI, err := uints.NewBytes(api)
if err != nil {
 return err
}

sha, err := sha2.New(api)
if err != nil {
 return err
}

calldataBytes := make([]uints.U8, 0, BatchSize*64)
for i := 0; i < BatchSize; i++ {
 calldataBytes = append(calldataBytes, variableToBytesBE(api, bytesAPI, circuit.Claims[i].Wallet)...)
 calldataBytes = append(calldataBytes, variableToBytesBE(api, bytesAPI, circuit.Claims[i].Balance)...)
}

sha.Write(calldataBytes)
digest := sha.Sum()

commitmentHi := digestLimbFromBytes(api, bytesAPI, digest[:16])
commitmentLo := digestLimbFromBytes(api, bytesAPI, digest[16:])

api.AssertIsEqual(commitmentHi, circuit.CommitmentHi)
api.AssertIsEqual(commitmentLo, circuit.CommitmentLo)

return nil
}
```

The commitment adds four proof words and a four-byte commitment-count prefix to the serialized proof:

```
Standard A/B/C proof:        8 × 32 = 256 bytes
Pedersen commitment:         2 × 32 =  64 bytes
Commitment proof of knowledge: 2 × 32 =  64 bytes
Commitment-count prefix:                4 bytes
Total:                                388 bytes
```

Proof generation therefore completes, but serialization fails with unexpected proof encoding length: 388. A commitment-aware verifier also expects separate commitment and proof-of-knowledge arguments, while the vault uses the following two-argument interface:

```
function verifyProof(
    uint256[8] calldata proof,
    uint256[6] calldata input
) external view;
```

As a result, proofs generated from the current batch circuit cannot complete the **claimBatch** path.

Remediation:

- Align the batch proof serialization, verifier interface, and vault call with the circuit's commitment-aware proof structure, or remove the compiler-introduced commitment from the circuit.
- Configure the batch prover and Solidity verifier to use the same commitment transcript.
- Regenerate the batch proving artifacts and add an end-to-end regression test covering proof generation, serialization, and **claimBatch** verification.

